IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 2 May 2025, accepted 27 May 2025, date of publication 2 June 2025, date of current version 9 June 2025.

*Digital Object Identifier 10.1109/ACCESS.2025.3575437*

RESEARCH ARTICLE

# PoolRunner: An Extensible Performance Testing Simulation Tool for Thread-Pool Middleware

**FAISAL BAHADUR[1], ZULFIQAR AHMAD[1], AND ABDULMOHSEN ALGARNI[2]**
[1]Department of Computer Science and Information Technology, Hazara University, Mansehra 21300, Pakistan
[2]Department of Computer Science, King Khalid University, Abha 61421, Saudi Arabia

Corresponding authors: Faisal Bahadur (faisal@hu.edu.pk) and Zulfiqar Ahmad (zulfiqarahmad@hu.edu.pk)

This work was supported by the Deanship of Scientific Research and Graduate Studies at King Khalid University under Grant R.G.P.2/21/46.

**ABSTRACT** Thread Pool System (TPS) is the most dominant concurrency control middleware service that boosts the scalability and performance of multithreaded servers. However, the performance evaluation of TPS is problematic. The performance of a TPS is tested at the end of the project by the load testing of the multithreaded server (in which the TPS is embedded) in a testbed environment. The TPS cannot be assessed alone early in the SDLC at the design time of the multithreaded server. This lacks the notion of early performance testing. Furthermore, the testbed construction and configurations are complex, difficult, time-consuming, and expensive, especially for researchers. Setting up the lab, learning testbed configurations, embedding TPS in an open-source multithreaded application (e.g. a web server or an application server), selection of load testing tool, and getting expertise to use the tool; all these are added costs for a researcher. Moreover, the load testing tools in the testbed environment cannot assess the internal performance metrics of TPS because the tool communicates only with the multithreaded server at the application layer. Thus, in this paper, we present the PoolRunner, a client-server-based extensible performance testing simulation tool for TPS, developed in JAVA. The PoolRunner is required not only by the practitioners in the industry in early performance testing when architectural choices are made for multithreaded servers, but also by researchers in the academia to assess and compare the performance of their new TPS with existing state-of-the-art TPSs. To the best of our knowledge, there is currently no publicly available simulation tool specifically designed for the early-stage performance evaluation of TPS. Our work contributes by developing a TPS-based performance testing simulation tool and by providing an extendible framework within the tool that enables researchers to embed and compare their custom TPS implementations with existing ones. As an example, we have embedded in the PoolRunner a state-of-the-art TPS of Oracle corporation as a baseline, as well as a static TPS for learning and validity. A detailed experimental evaluation of PoolRunner validates the correctness, robustness and high performance of PoolRunner.

**INDEX TERMS** Thread pool system, performance testing, distributed system, PoolRunner, simulation, scalability, multithreaded servers.

## I. INTRODUCTION

Scalability is one of the utmost non-functional requirements of server-side applications [1], [2] because it increases performance proportional to the resources added to the system on high loads [3], [4]. It is one of the most critical challenges and a key design requirement of network-based server applications [5], [6], [7]. The scalability of server-side applications is entirely dependent upon its middleware architecture, which means that the middleware of server must be designed with a view to scaling [8], [9]. A high scalable server-application can be achieved only if designers and software engineers consider the scalability of its middleware at the initial stages of SDLC when architectural choices

The associate editor coordinating the review of this manuscript and approving it for publication was Ghufran Ahmed.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

are made. Thus, middleware architecture makes server-side applications scalable, highly available and highly performant [10], [11], [12], [13].

The most significant architecture used by the middleware to improve performance and scalability is TPS [14], [15]. TPS has been used frequently as a middleware concurrency control service in plethora of server-side applications including web servers, application servers, distributed systems, ultra-large-scale systems, cloud environments and IoT systems [16]. It has not only been used in server-side applications but also been used in many other applications where real time demand is critical for processing requests/events [12].

The key concern in TPS is the dynamic optimization of thread pool size that determines the overall performance of TPS and improves quality of service [17]. The TPS-based systems are overloaded either due to smaller or larger thread pool sizes, instead of adopting an optimal one. Experimental studies have proved that the most significant factor that affects the overall performance of application servers is the size of thread pool [18]. Determining the optimal size of thread pool dynamically on runtime is a challenging problem [19]. To tackle this problem, the number of research contributions in the last two decades have presented different TPS optimization strategies, but here we are referencing some of the latest from 2019 to 2025 [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30]. The diversity of target servers where TPSs are deployed makes dynamic optimization more challenging, as there are different characteristics of the deployment system with a different nature of tasks, thus the TPS has evolved from vertically scaled TPS into horizontally scaled TPS, also called distributed thread pool systems.

As said earlier, the middleware used to build a server application usually determines the overall performance of the application. However, the middleware testing itself is not without challenges, because testing the internals of something designed as an encapsulated system is very difficult. Technology makers and s/w developers in the industry construct multithreaded servers on the top of TPS-middleware, whereas researchers in the academia work on dynamic optimization of TPS to provide more performance efficient TPS. From both perspectives of industry and academia, a performance testing simulation tool specific to TPS is required; for the industry to support the notion of early performance testing; and for the academia to easily assess and compare different TPS's performance. Presently, the TPS-based applications are tested on the testbed environment after project completion. The construction and configurations of testbeds and labs are complex, difficult, time consuming and expensive specifically for researchers. Setting the lab, learning testbed configurations, embedding TPS in an open-source multithreaded application (e.g. a web server or an application server) and selection of load testing tool and getting expertise to use the tool; all these are added costs for a researcher. Furthermore, in the testbed environment, the commercial performance testing tools are unable to assess the internal performance metrics of TPS, as these tools communicate with the end-product and not with the TPS which is embedded inside the end-product as a middleware [31], [32]. Consequently, these tools assess only external or application-level performance metrics such as response latency and throughput, which are often affected by external noise including network delays, client-server distance, and background system load. In contrast, TPS-level internal measurements remain insulated from such disturbances, allowing for more accurate and reliable performance evaluation. External metrics provide only a surface-level view and are insufficient for understanding the internal performance dynamics of middleware components (e.g. TPS). Moreover, due to the unavailability of a TPS-specific simulation tool, one cannot assess and select a TPS early in SDLC at design time of target application that can provide the required performance to the target application. A common misapprehension among designers and developers of the software is to take the functionality as a key factor of the product, while considering performance and scalability after the s/w product is developed. The s/w projects that are built around this misconception fail and put the companies at risks. Furthermore, the same middleware may perform very differently in the context of different applications. For example, a TPS deployed for an email server can give different performance than deploying it in a web server, as both target applications have different characteristics. Hence, it is desirable that the TPS is first tested at early development stages of target application by some state-of-the-art simulation tool that can assess the TPS with testing parameters specific to the target application [28], [33], [34].

To overcome these challenges of the TPS performance testing, this paper presents PoolRunner, a client-server based extensible performance testing simulation tool for TPS that is developed in JAVA. The PoolRunner can be used in the industry early in SDLC at design time of target application to first evaluate different TPSs, and then select a proper one for target application. It is also a remedy to the complex testbed environments for researchers. It provides an extensible framework, that allows researchers to add a new TPS in the system and make it available to others for comparison and validity. The PoolRunner can easily assess all the internal performance metrics of TPS. In this paper, we highlighted the requirements that must be present to develop a state-of-the-art extensible simulation system for TPS. Furthermore, we have embedded a state-of-the-art TPS of Oracle corporation called ThreadPoolExecutor [35] in the simulation system as a baseline, as well as a static TPSS for learning and validity.

The remaining sections of this paper are organized as follows.

In section II we present the related work, section III provides requirements for a performance testing simulation framework for TPS. The detailed architecture of the PoolRunner is given in section IV. Section V presents the performance

validation of PoolRunner and finally sections VI and VII conclude the paper with limitations and future work.

## II. RELATED WORK

In this section we will go through popular open-source performance testing tools that can be indirectly used to test the performance of TPS.

### A. JMH (JAVA MICROBENCHMARKING HARNESS)

This is a popular tool designed for benchmarking the performance of Java code including algorithms, data structures, and thread pool systems, with a focus on accuracy and precision [36]. It is widely used for measuring external performance metrics such as latency (time taken to process a single task) and throughput (tasks completed per second) for micro-level operations. However, JMH is not a simulation tool, meaning it doesn't model or simulate complex behaviors or hypothetical system states. It strictly measures real performance based on the actual execution of code under the JVM's conditions. While JMH excels in benchmarking the performance of Java code, it presents some limitations when it comes to testing the performance of thread pool systems. One significant limitation is that JMH does not provide visualizations or built-in support for measuring internal metrics of thread pool systems, such as pool size, request wait times in the queue, or the response times of requests inside the thread pool. Researchers wanting to assess such metrics in JMH would need to manually track and log them within their benchmark methods, which can become cumbersome and error-prone. Additionally, it is command-line based and lacks a graphical user interface (GUI). This means that tester must analyze the results in raw format (e.g., text, CSV, or JSON) and utilize external tools (like Grafana or Excel) for visualization. Regarding extensibility, JMH does not offer built-in support for reusable thread pool modules. Each benchmark project is self-contained — you can't globally "register" a thread pool for other testers unless they reuse your codebase. Moreover, JMH doesn't provide automatic comparative analysis.

Although JMH is a robust tool for measuring external performance metrics, it is not ideal for comprehensive thread pool performance testing. Its lack of internal metric tracking, reliance on manual logging, absence of visualization features, and command-line interface make it less suited for researchers focused on deep thread pool analysis.

### B. JCSTRESS

JCStress is a tool primarily designed for detecting subtle concurrency issues in Java programs, especially those involving multi-threading [37]. It focuses on ensuring the correctness of concurrent code by identifying problems like race conditions, visibility, and atomicity, rather than measuring performance metrics. When it comes to thread pool performance testing, JCStress can be used indirectly to verify that your thread pool implementation handles concurrency correctly under stress. However, it is not designed for measuring traditional performance metrics like latency, throughput, or internal thread pool metrics (e.g., pool size, wait times). JCStress is command-line based, offering a simple yet effective way to test the correctness of concurrent operations. It provides basic results, such as pass/fail conditions, which help identify concurrency bugs but lacks built-in functionality for measuring thread pool performance or providing in-depth insights into thread pool behavior. Integration with a custom thread pool system typically requires modifications to the code, as JCStress expects specific test scenarios and doesn't offer built-in support for external TPS configurations. Additionally, JCStress doesn't have a GUI or built-in visualizations. The results are presented in plain text, and any graphical representation or deeper analysis requires exporting the data to external tools. It doesn't provide real-time monitoring or dashboards, and users manually log internal metrics if they wish to gather data beyond the tool's core functionality. JCStress excels at concurrency correctness but falls short in being a tool for performance benchmarking or deep thread pool system analysis.

### C. LDBC'S TPSTEST

In "LDBC's TPSTest", the term TPS stands for Transactions Per Second. TPSTest is a simulation tool developed by the Linked Data Benchmark Council (LDBC) to evaluate the transactional throughput (TPS) of database systems under controlled workloads [38]. TPSTest can be used to observe the effects of thread pool configurations indirectly during workload execution, however it does not provide built-in support for embedding or evaluating custom thread pool systems. To integrate a new thread pool implementation, developers usually must modify the internal source code, especially the parts managing task execution and workload dispatching. This makes TPSTest less extensible. TPSTest is not GUI-based. It is a command-line tool and works by running predefined benchmark workloads. Results are typically presented in the console, though they can be output to text files for later analysis. You would need external tools (like Excel, Grafana, etc.) for graphical analysis of the results. TPSTest is designed to measure high-level performance metrics like throughput and latency. While it can test how different thread pool sizes affect these metrics, it does not provide built-in functionality to track internal thread pool metrics like pool size changes, wait times, or queue lengths. We need to manually log these metrics within the thread pool system and integrate them with TPSTest for tracking. Although this tool can be used to compare the performance of different thread pool configurations, it won't directly provide a comparison output. You would need to manually interpret the results.

### D. ADAPTATIONS OF YCSB

Adaptations of YCSB are quite famous in benchmarking research, but originally YCSB was made to test databases and cloud storage systems — not thread pool systems [39].

YCSB is a command-line based tool, not a GUI tool, and it is a pure benchmarking tool, not a simulation tool. If a researcher wants to use YCSB for thread pool testing, he will have to do a lot of adaptation, because YCSB expects database read/write workloads by default. Adaptations of YCSB do not provide built-in thread pool extensibility. Integrating a custom thread pool typically requires modifications to the core client execution logic within the source code. You need to first create a proper adapter for your new thread pool system and register it inside it, then other researchers can also add their thread pools and compare performance results later. But again, the researcher will have to manually run tests and manually compare the results — YCSB does not automatically declare a ''winner.'' So, overall, YCSB can be adapted for thread pool testing if needed, but it is not a natural or easy fit. Some effort is required. YCSB does not measure internal thread pool metrics like pool size, queue waiting time, or internal response time. It only measures external metrics like throughput and request latency. You can save results to a file and later create graphs manually using some external tool, but YCSB itself does not provide a built-in GUI or live graphs.

### E. APACHE JMETER

Apache JMeter is a very popular load testing and performance testing tool. It is GUI-based and user-friendly for creating and running load tests [40]. JMeter is a simulation tool because it simulates heavy loads on servers, databases, or websites to see how they behave under pressure. However, when it comes to direct testing of thread pool systems (like ThreadPoolExecutor or custom thread pools), JMeter is not a natural choice. JMeter does not go inside a thread pool to measure internal metrics like pool size, thread wait times, or internal throughput. It only measures external results like request completion times. If you really want to access internal thread pool metrics, you must modify your system to expose that data externally and then somehow capture it through JMeter, which is not easy. JMeter does not offer built-in extensibility for embedding custom thread pool implementations. Custom thread management requires creating plugins or modifying test elements using JMeter's extension APIs or source code which is very hard and time consuming. On the positive side, JMeter does provide very good graphs, dashboards, and visual reports, which makes result analysis easier compared to tools like JMH. But still, when you want to compare two different thread pools, you must manually run the tests separately and compare the results yourself — JMeter will not automatically tell you which thread pool performed better. In short, JMeter is very good for full application load testing, but it is not a specialized tool for thread pool performance research.

### F. GATLING

Gatling is a load testing tool widely used for evaluating the performance of web applications and APIs [41]. Although not specifically designed for thread pool performance testing, it can be adapted to such tasks with some custom configuration. The tool offers a user-friendly graphical interface, making it easier for testers to design and run complex load tests without a deep dive into command-line configurations. This GUI-based design is complemented by its ability to generate visual graphs, such as real-time performance charts and reports, which can be useful when analyzing the overall throughput and response times of requests handled by thread pools. However, Gatling does not directly measure internal thread pool metrics like pool size, thread wait times, or internal response times. For thread pool performance testing, such metrics would need to be logged manually, with some effort to integrate custom logging mechanisms into the system being tested. Gatling does not have built-in extensibility specifically for thread pool systems (TPS). While it is highly customizable for various testing scenarios, integrating and managing custom thread pools typically requires manual configuration or custom code within its simulation framework. While Gatling can simulate high loads on a system, it does not offer built-in functionalities for comparing different thread pool configurations automatically. Testers would need to manually analyze results and make comparisons based on throughput, request count, and response times. Gatling can store results in various formats, including HTML, CSV, and JSON, which are easy to work with for post-test analysis. It also provides real-time dashboards during test execution, allowing testers to monitor performance continuously. Primarily, Gatling is a load testing tool that excels in stress testing web applications and APIs. It runs on the JVM and is compatible with all major platforms, including Windows, macOS, and Linux. Although not initially intended for thread pool testing, its flexibility makes it a strong candidate when adapted to this use case.

### G. APACHE BENCH

Apache Bench, or ab, is a command-line tool that is typically used for load testing HTTP servers [42]. It is a simple, lightweight benchmarking tool focused on measuring external performance metrics like throughput, response times, and request completion times. While Apache Bench is not specifically designed for thread pool system performance testing, it can still be used indirectly to gauge how well a system with a thread pool manages incoming requests under heavy load. However, it has several limitations when used for this purpose. Apache Bench does not provide any internal metrics related to thread pools, such as pool size, queue wait times, or internal response times. It only measures the external aspects of the system, primarily focusing on how long it takes to complete a series of HTTP requests. For thread pool performance testing, this means that users would not gain insights into how the thread pool is functioning behind the scenes unless they manually log these internal metrics themselves. Moreover, Apache Bench lacks any extensibility

**TABLE 1.** Comparative analysis of benchmarking and simulation tools with respect to thread pool system (TPS) performance testing features.

| Tool | JMH | JCStress | TPSTest (LDBC) | YCSB Adaptations | JMeter | Gatling | Apache Bench |
|---|---|---|---|---|---|---|---|
| **GUI Based?** | No | No | No | No | Yes | Yes | No |
| **Visual Graphs?** | No | No | No | No | Yes | Yes | No |
| **TPS Internals Measurement?** | No | No | No | No | No | No | No |
| **Designed for TPS Testing** | No | No | No | No | No | No | No |
| **Built-in Extensibility in terms of TPS?** | No | No | No | No | No | No | No |
| **Tool Type** | Benchmarking tool | Benchmarking tool | Simulation Tool | Benchmarking tool | Simulation Tool | Simulation Tool | Simulation Tool |
| **Automatic TPS Comparison?** | No | No | No | No | No | No | No |
| **Saving Strategy?** | Saves in JSON, text, CSV | Saves basic results (text) | Test result reports (CSV) | Result files (text/JSON) | Visual reports, CSV/XML | Saves in files (CSV, JSON) | Saves in text files |
| **Key Metrics Measured** | Latency, Throughput | Concurrency correctness (e.g., race conditions) | Latency, Throughput | Database read/write latency and throughput | Response time, Request count, Failures | Latency, Throughput, Requests/sec | Request count, Response time |
| **Ease of Integration with Custom TPS** | Moderate | Low | Low | Moderate | Low | High | Low |
| **Real-time Monitoring Capability (Dashboard)** | No | No | No | No | Yes | Yes | No |
| **Primary Domain Focus** | Microbenchmarking | Concurrency correctness | Transactional throughput | Database load testing | Full application load testing | Web and application performance testing | Simple web performance testing |
| **Platform / Language Dependence** | Java | Java | Java | Java | Java, Multi-language support | JVM-based (Scala) | Command-line, Cross-platform |

for integrating custom thread pool implementations. It is a basic tool that operates on a very simple principle of sending multiple HTTP requests to a server and measuring the time it takes to process them. It cannot be easily adapted to test the internal workings of a custom thread pool system. There is also no built-in feature for real-time monitoring or graphical reports. Apache Bench simply outputs the results in plain text, which can be saved and analyzed later using external tools like Excel or Grafana. It's widely used for basic HTTP load testing but not ideal for deep thread pool performance research. For users who need to examine the behavior of thread pools, Apache Bench offers limited insight and requires substantial manual work to capture internal metrics. Overall, while Apache Bench is a great tool for HTTP server load testing, its suitability for thread pool performance testing is limited, and adapting it to such tasks would require considerable effort.

To evaluate the suitability of existing tools for TPS performance testing, a detailed comparison is conducted across all these tools. Table 1 summarizes the capabilities of each tool with respect to various key features relevant to TPS evaluation. Based on this table none of the presented tools collectively offer:

1. Native support for internal TPS metrics (e.g., live pool size, queue length, thread wait times),
2. Automatic comparison of different TPS variants (e.g., testing multiple strategies under same workload),
3. TPS-Extensibility for researchers (e.g., easily extending the tool by adding new TPSs for comparisons).

As of now, there is no widely known or established tool that is specifically and exclusively designed for thread pool system (TPS) performance testing — in the way that, for example, JMH is specific for microbenchmarking, or JCStress is for concurrency correctness, JMeter for load

**TABLE 2.** Key requirements of a performance testing simulation framework.

| Identity | Requirement | Description |
|---|---|---|
| R#1 | Extendibility | Ability to allow a user to dynamically port a new TPS at run time without changing the application code base. |
| R#2 | Workload Reality | Ability to generate both static and dynamic requests of different sizes (service times) |
| R#3 | Load Reality | Ability to generate request rates with different probability distributions. |
| R#4 | Open system model | Ability to submit requests at the specified rate without waiting for the completion of previous requests. |
| R#5 | Measurability | Ability to measure all internal performance metrics of TPS including response time, queue wait, pool size, throughput. |
| R#6 | Data dynamicity | Ability to calculate the performance metrics dynamically, without consuming large amount of memory and without using background disk access (to avoid interference and delay in simulation experiment). |
| R#7 | Data Saving | Ability to save the test report as well as test plan for later use in analyzing it and comparing it with other TPSs. |
| R#8 | Comparability | Ability to perform comparative analysis of performance between different TPSs. |

simulation, YCSB for database testing, TPSTest (LDBC) for transactional throughput analysis, Gatling for load testing web applications, and Apache Bench (ab) for simple HTTP load testing. Researchers either adapt any of these general-purpose tools to indirectly test TPS or build their own minimal test frameworks for experiments. There is no mainstream, dedicated TPS performance testing tool. Relying on these general tools for TPS testing is akin to using a spreadsheet to model a startup's financial structure: while technically possible, it lacks domain-specific templates and requires substantial customization. Similarly, TPS researchers must develop wrappers, inject instrumentation, or build ad hoc test frameworks around these tools to get meaningful insights — a process that is inefficient and error-prone.

This leads us to the central research question: How to build a state-of-the-art, dedicated TPS performance testing simulation framework that can natively support TPS-internal-metrics, automatic comparison of TPS variants and built-in extensibility in terms of TPS. To address this, we present PoolRunner in this paper.

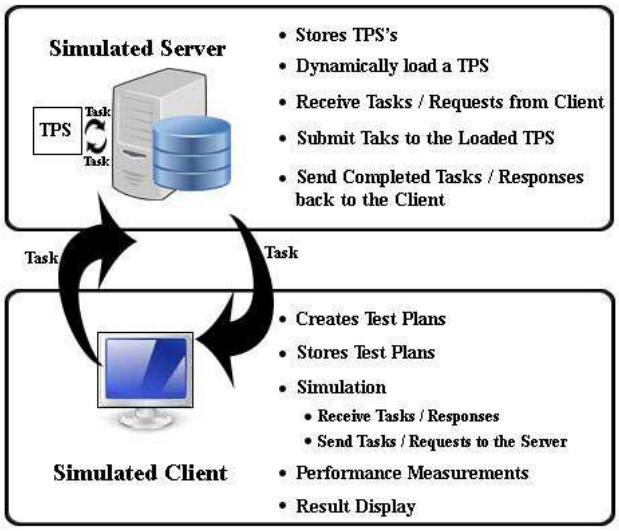



**FIGURE 1.** PoolRUnner system architecture.

## III. REQUIREMENTS FOR A LOAD TESTING SIMULATOR

Before architecting and developing a performance testing simulation framework for TPS, requirements need to be framed which the framework shall address. *Table 2* details the performance testing simulation framework for TPS [43], [44], [45]. These requirements are certainly not exhausting; however, we only cover central aspects. In section III, we then discuss how PoolRunner fulfills these requirements.

## IV. SYSTEM ARCHITECTURE

The PoolRunner application uses a client-server architecture shown in Figure 1, where a user will first embed a TPS in the simulated server.

Next, the user will use the interface of simulated client to connect to the simulated server and evaluate TPS running in the simulated server. The simulated client will send requests to the simulated server and receive server's responses. The simulated server behaves as a multithreaded server to the TPS that receives client's requests and submits those requests to TPS for execution. TPS hands over the processed requests to the simulated server in the form of responses that sends the responses back to the client.

The requests are simulated by an object of class Task shown in Figure 2. The IoBoundTask and CpuBoundTask classes that extend the Task class simulates two different types of requests, static request and dynamic request. The Task objects are Runnable and Serializable. Runnable means they can be executed by Threads, and Serializable means they can be converted into stream of bytes and sent over the network. A Task has a set of time-based parameters that are set by the simulated server during its lifetime in TPS. These parameters are used to calculate waiting latency and response time of each Task.

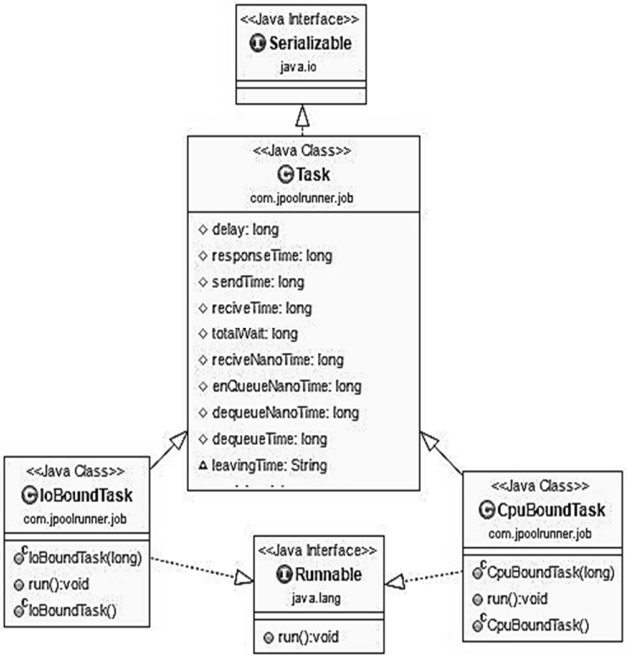



**FIGURE 2.** Task 's class diagram.

### A. SIMULATOR ARCHITECTURE AND CAPABILITIES

The simulator consists of two tiers, the client –tier and the server-tier where each tier further consists of number of components. In the next section we will explore each tier separately.

#### 1) SERVER-TIER

The architecture of the server-tier is given in Figure 3. When the server-tier starts, it initiates the connection manager that listens for the client's connection on server socket. When a client connects with the server through socket, the connection manager starts another thread component called service-name communicator that first fetches the names of existing TPS's through extensibility framework. TPS's are available as services on the server-tier and extensibility framework is responsible for adding a new TPS, to fetch the names of available TPS's and to dynamically load TPS for its performance evaluation. The service name communicator sends the names of available TPS's to the client through socket's output stream and waits for the client's request to start a TPS on the server for performance evaluation. When the name of a TPS is sent by the client, the Service-Name communicator sends a signal to the extensibility framework to load the TPS and start it. The TPS starts its execution, and it is ready to run the Tasks sent by the client. Finally, the service-name communicator starts with Receiver, Consumer and Response_Sender threads and finishes its execution. The Receiver is responsible to receive client's Tasks and enqueue them in a producer-consumer queue. There may be thousands of Tasks send by the client per second, and all these tasks are submitted to the queue. Every Task is time-stamped in

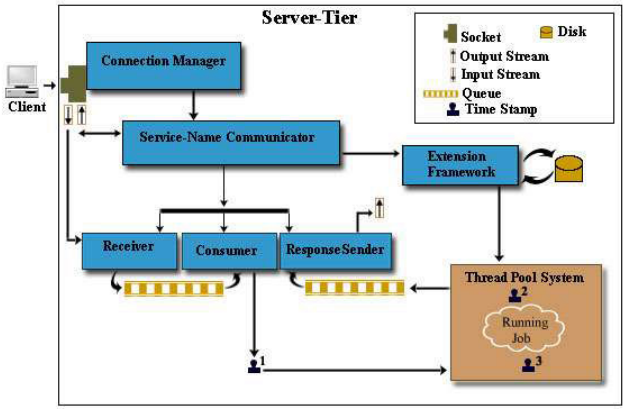



**FIGURE 3.** The architecture of server tier.

milliseconds on the server side at different stages to assess the TPS performance. The Consumer repeatedly dequeues each Task from the queue, time stamps it (timestamp1) and submits the task to the loaded TPS. The TPS will save it in a Task queue where it will wait for a free thread in the pool to run it. When a Task is picked by a free thread in the TPS, it is again marked with the second stamp (timestamp2) and finally the thread starts its execution. When a thread finishes the Task, it is again marked by third time stamp(timestamp3). The finished Task (response) is then enqueued in a queue. The Response_Sender thread de-queues the Task and sends it back to the client on output stream as a response by the server. When the Receiver reads an end of simulation signal by the client from input stream, it sends this signal to the Consumer and Response_Sender and ends it execution. Meanwhile, it is possible that there may be thousands of requests in the request queue waiting to be consumed by the Consumer. Hence, the Consumer delays its exit until it dequeues all the Tasks in the request queue and submit these tasks to the TPS. Similarly, the Response_Sender dequeues all the Tasks from response queue and sends those tasks back to the client. Finally, before exiting, the Response_Sender restarts Service_name Communicator thread again so that the client can pick a new TPS for performance testing. The following two equations are used to assess the wait time and response time of each Task on the client-tier.

Waiting Latency=timestamp2-timestamp1 (5)

Response Time=timestamp3-timestamp1 (6)

#### a: EXTENSIBILITY/EXTENSION FRAMEWORK

The PoolRunner is an extensible application in such a way that a new TPS can be added in it on run time in the form of a jar file without changing the code base of PoolRunner. Once added, the TPS becomes a part of the PoolRunner software. The PoolRunner uses Oracle's solution of Service Provider Interface (SPI) [46] to provide an extensibility mechanism. The PoolRunner provides an extensible service called TPS_Service that is shown in Figure 4. The TPS_Service locates and loads available TPSs

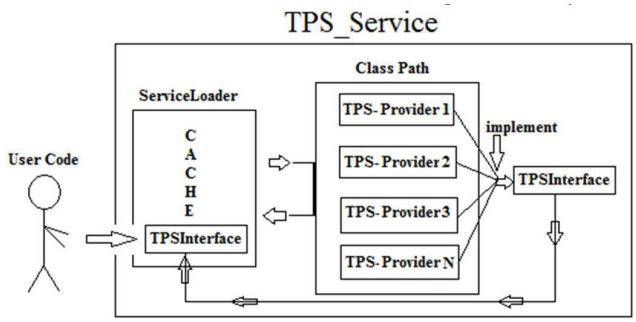



**FIGURE 4.** PoolRunner TPS_service.

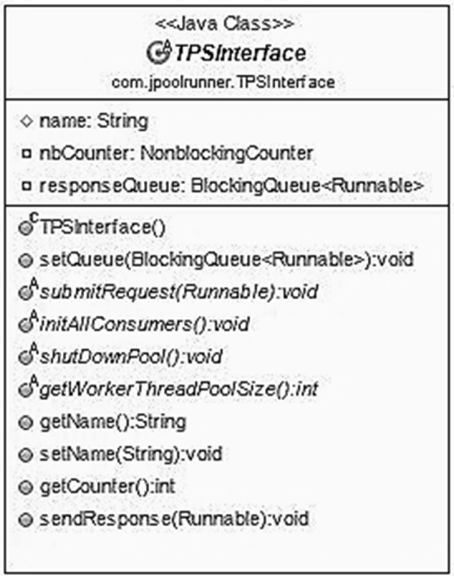



**FIGURE 5.** PoolRunner TPSInterface class.

from its class path. The PoolRunner's SPI mechanism works in the following way. The TPS_Service has defined a contract named TPSInterface. The TPS providers will implement this interface and provide implementation of their new TPS. The TPS providers will store their new TPS in TPS_SERVICE. A new TPS that is developed by a researcher or practitioner is basically a TPS provider. Hence, the new TPS must implement TPSInterface and store in the TPS_Service. As discussed earlier, that an SPI may be an abstract class or an interface, similarly, the TPSInterface is an abstract class. The class diagram of TPSInterface is shown in Figure 5, that has four abstract methods that must be overridden by the TPS-Providers to provide their implementation. Table 3. summarizes these 4 abstract methods and one non abstract method. We have embedded in PoolRunner a state-of-the-art TPS of Oracle corporation (ThreadPoolExecutor [28]) as a TPS-Provider, as well as a static TPS for learning and validity.

### 2) CLIENT TIER

The architecture of Client-Tier is given in Figure 6. The functionalities of the client tier consist of workload

**TABLE 3.** Description of the important methods of TPSInerface.

| Method Name | Description |
|---|---|
| abstract submitRequest(Runnable):void | An abstract method called by the simulated server from Consumer to submit a Task to TPS. |
| abstract initAllConsumers():void | An abstract method called by extension framework to initialize a TPS for performance testing. This method should initialize a particular number of threads in the pool. |
| abstract getWorkerThreadPoolSize():int | An abstract method that returns the current number of threads in the pool |
| abstract shutDownPool():void | An abstract method that shutdowns the TPS. This method should stop all threads present in the pool. |
| sendResponse(Runnable):void | Non abstract method that must be called by the TPS-Provider when a CPU or I/O bound task is finished. Just pass the finished Task to this method so that server can send finished Task back to the client |

construction, load generation by specific distribution, receiving server responses, plotting responses in the form of different graphs of performance metrics, and displaying comparative analysis results in graphs. The Timer is used to display the elapsed time of a simulation. When the user stops simulation the Client-Tier waits until it receives the last response from the server. Hereafter, the Timer stops.

#### *a: TEST PLAN*

To test a TPS, the user creates a test plan that specifies the workload characteristics and a particular load generation engine. The load generation engine will create request frequencies at a particular distribution that represents pattern of request arrival on the server. During simulation, the generated workload, request frequencies and received responses are collected in the data structures for performance measurement and comparative analysis. Each instance of a test plan has its own TPS to test, generate workload, request frequencies, collected responses and GUI. In the case of comparing two TPSs, the saved workload and request frequencies of already tested TPS are reused to test other TPS. When a TPS has been tested, it can be compared with any number of available TPSs on the server.

#### *b: WORKLOAD CHARACTERIZER*

This component allows a user to create a workload with the help of a GUI. There are two types of tasks (static and dynamic) available to simulate the workload. First the user specifies the intensity (percentage) of static and dynamic

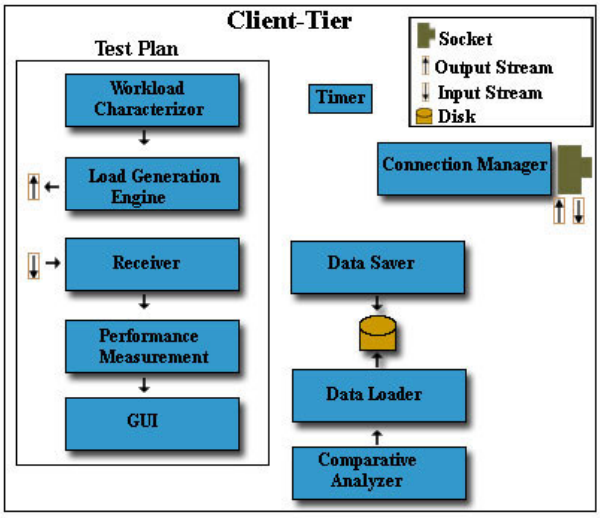



**FIGURE 6.** PoolRunner client tier architecture.

requests in the workload. Next, the user selects the types (sizes) of static and dynamic files. Finally, the user specifies the access frequency of each file.

The IoBoundTask and CPU-Bound classes that extend the Task class simulates two different types of requests, static request and dynamic request. The static files are simulated by IoBoundTask request, and the size of static file is simulated by the amount time the thread running the IoBoundTask request runs (sleeps). The percentage of static files in the static workload is specified by access frequency. Table 4. is a list of static files with corresponding sleep times and default access frequencies. The access frequencies can be changed according to the user's need. The granularity of sleep time is assured by the interruption period of thread scheduler, which is 15ms in windows and 1ms in Linux. Hence, we cannot take very small sleep times. We have used minimum amount of sleep to 100ms. We selected four types of static files with various size ranges and various default access frequencies in our static workload. We adopted static file sizes and access frequencies from standard benchmarks, WebStone [47] and SPECweb99 [48] load generation tools. CpuBoundTask is simulated by keeping the processor busy for a specific amount of time. When a CpuBoundTask Task runs, its corresponding thread generates specific number of random numbers depending upon intensity of service time.

Table 5. summarizes the types of CpuBoundTask used in application. Pure CPU-bound workload is used to test only distributed thread pool that is running on the cluster. Testing a TPS or distributed TPS on the local host should include a small CPU-bound workload, otherwise application performance is affected.

### c: LOAD GENERATION ENGINE (REQUEST RATE GENERATION)

The load generation engine sends the prepared workload to the server at specific rate defined by specified probability distribution. After every second it generates a random number based on the specified probability distribution, and based on that number the amount of workload is sent to the server. For example, if the random number is 100, then 100 requests are sent to the server based on specified workload characteristics. There are five types of probability distributions used to generate random number. Table 6. lists these distributions with associated parameters. Simulator sends the tasks at a specified rate by open system model. Each Task represents a request by a single client and the number of concurrent clients may be increase by increasing the request rate. The Figure 7 is a class diagram that shows that the Load Generation engine is a Timer class having different probability distributions.

**TABLE 4.** Static files information.

| No | File Size (kb) | Service Time/Sleep Time | Default Access Frequency |
|---|---|---|---|
| 1 | 1 | ≈ 100ms | 30% |
| 2 | 10 | ≈ 200 ms | 50% |
| 3 | 100 | ≈ 300 ms | 20% |
| 4 | 1000 | ≈ 500 ms | 0 |
| 5 | 2000 | ≈ 1000 ms | 0 |

**TABLE 5.** Dynamic request information.

| No | Task Type | Service Time |
|---|---|---|
| 1 | Low CPU-Bound | ≈ 40 ms |
| 2 | High CPU-Bound | ≈ 100ms |

**TABLE 6.** Load distribution methods available in the PoolRunner.

| No | Name | Parameters |
|---|---|---|
| 1 | Poisson Distribution | Average Request rate. i.e. λ |
| 2 | Gaussian Distribution | Mean, Standard Deviation |
| 3 | Uniform Distribution | Average Request rate |
| 4 | Constant Distribution | A constant number. |
| 5 | Heavy Tailed Distribution | Bursty Bigger Workload |

### d: PERFORMANCE METRICS

In this section, we will discuss the performance metrics that are assessed and shown by the simulator.

#### I) THROUGHPUT

This is the response rate per second. During simulation, the throughput per second is shown in xy-graph to portray the relationship between request rate and response rate as shown in Figure 8. At the end of the test, the total and average throughput is also shown to the user.

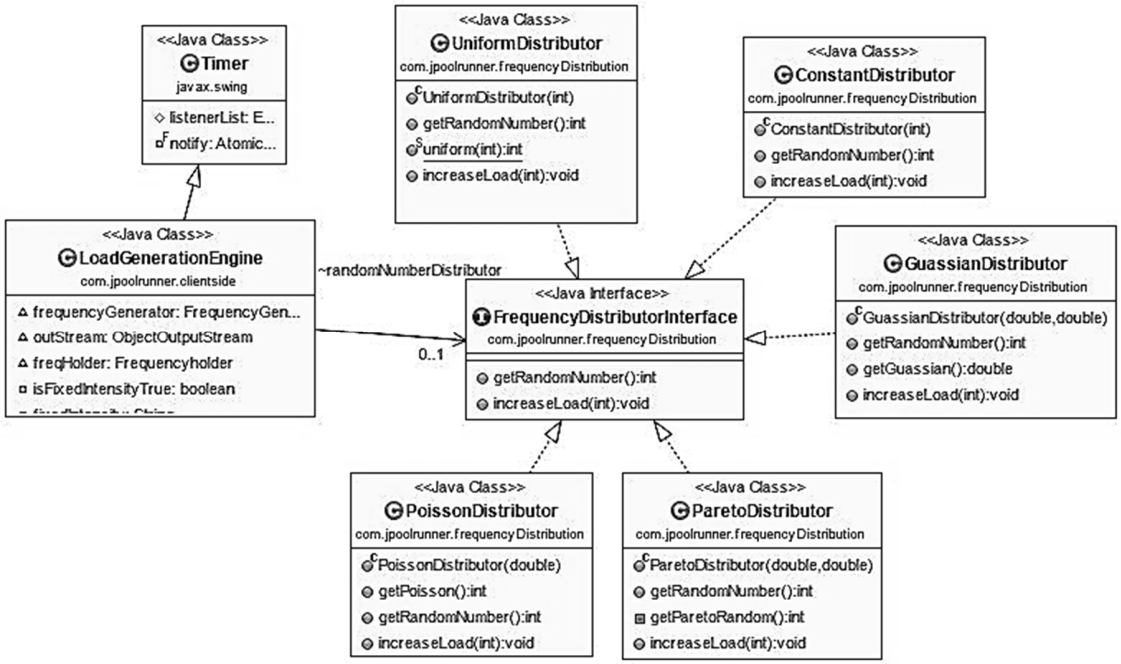



**FIGURE 7.** Class Relationships diagram of load generator.

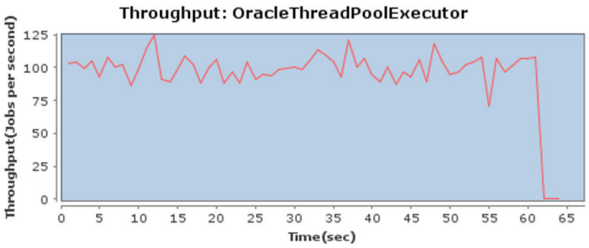



**FIGURE 8.** Throughput per second of a thread pool on runtime.

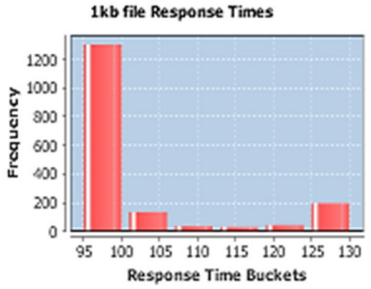



**FIGURE 9.** The runtime response time histograms of 1kb file.

### *II) RESPONSE TIME AND WAITING LATENCY*

Showing the response time and wait time (of each request) on the xy-graph during simulation requires huge amount of memory, as there may be thousands of responses arriving per second, depending upon the request rate. Therefore, PoolRunner shows response times and wait times information by histograms. There is a separate histogram for each file type in the workload. The histograms provide effective synopses of large data sets and are thus used in a wide variety of applications. Histograms summarizes the data into a smaller number of ''buckets'': the buckets only retain the information how many elements fall between two consecutive bucket boundaries, but not their precise values. On the arrival of a response, its corresponding histograms of response time and wait time are updated, by incrementing the counter of the bin in which the response time and wait time of the response falls in. Similarly, we used approximate quantiles [35], [36] for response times and wait times. The method to compute approximate quantiles from histograms works as follows.

Let the range of values be partitioned into k intervals. These intervals are known as bins. A single pass is made over the data and the count of values in each bin is accumulated. The bins are scanned from lowest to highest values and the quantiles are determined based on the counts in the bins. For each type of file, the PoolRunner shows the approximate $50^{th}$ and approximate $90^{th}$ percentiles by identifying the bin range in which a percentile falls in.

Figures 9,10 and 11 shows an example of the response time Histogram of 1kb, 10kb and 100 kb file on runtime.

Figure 12 shows the percentile statistics of 1kb, 10 kb and 100kb files at the end of the Test.

### *III) POOL SIZE*

This is the number of threads in the thread pool under testing. It is shown after every second in a xy-graph as shown in Figure 13. The Thread Pool under testing adjusts its size by its own dynamic tuning strategy over time.

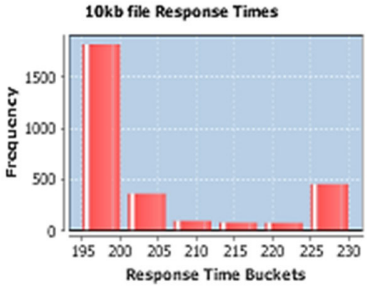



**FIGURE 10.** The runtime response time histograms of 10 kb file.

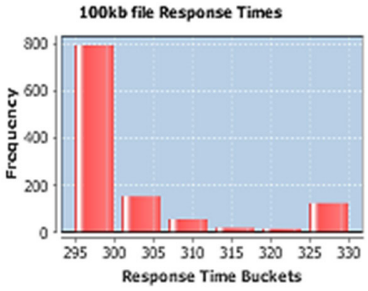



**FIGURE 11.** The runtime response time histograms of 100kb file.

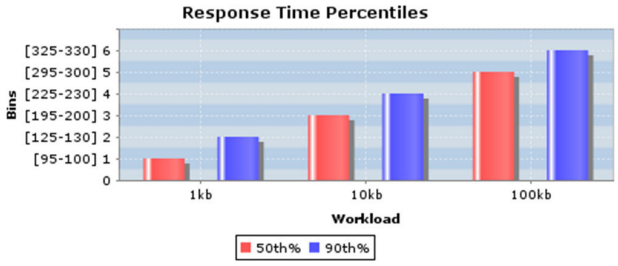



**FIGURE 12.** Percentile statistics of 1kb, 10 kb and 100kb files at the end of test.

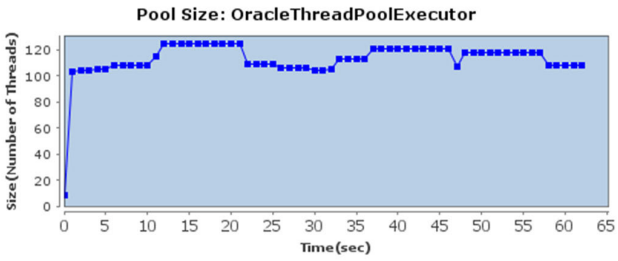



**FIGURE 13.** Thread pool size on runtime after every second.

### e: COMPARATIVE ANALYSIS

The performance of a TPS can only be compared with any other TPS available in the TPS_Service if the TPSs have run against the same workload and request frequencies. For this purpose, the dynamic workload and request frequencies of one TPS are recorded and re-used for the load testing any other TPS. The comparative analysis is presented in the form of graphs, and each performance metric is compared on a separate graph as shown in the Figures of this section. To show comparative analysis graphs of each performance metric, we first tested a static thread pool with 100 threads under the workload of 1000 request rate of Poisson distribution along with file sizes 1kb, 10kb and 100kb. The load on the server is shown in Figure 14. The same workload is used to test the dynamic ThreadPoolExecutor [28] that changes its pool size based on the incoming workload. The following are the comparative analysis graphs of different performance metrics shown in Figures 15 to 20.

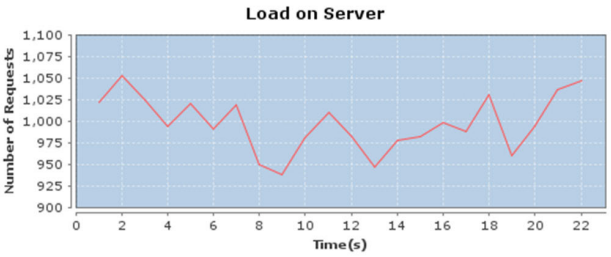



**FIGURE 14.** Load generated by the PoolRunner on TPS.

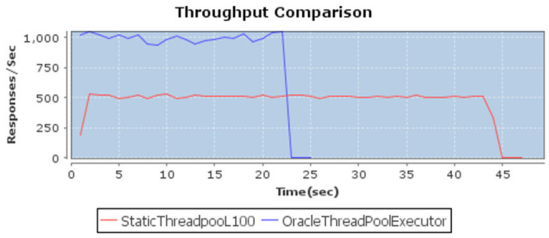



**FIGURE 15.** Comparative analysis of throughput between two different thread pool systems.

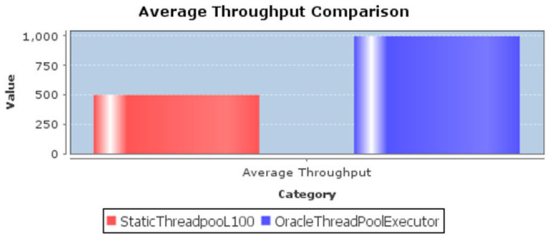



**FIGURE 16.** Comparative analysis of average throughput between two different TPSs.

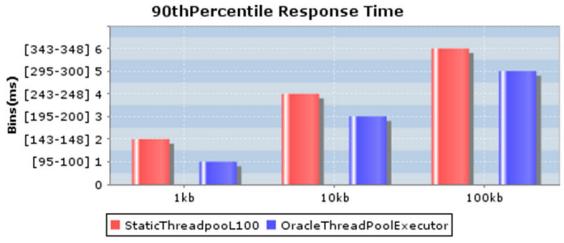



**FIGURE 17.** Comparative analysis of 90th percentile response times of each workload type.

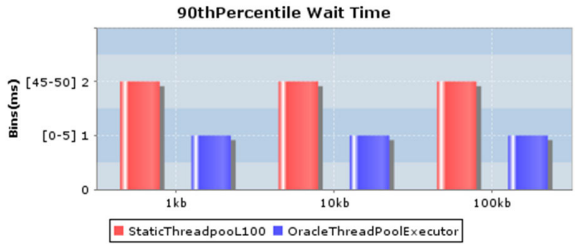



**FIGURE 18.** Comparative analysis of 90th percentile wait times of each workload type.

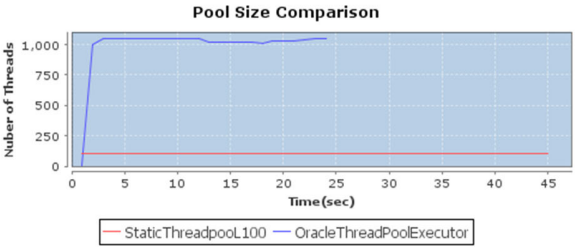



**FIGURE 19.** Comparative analysis of pool sizes of two different TPSs.

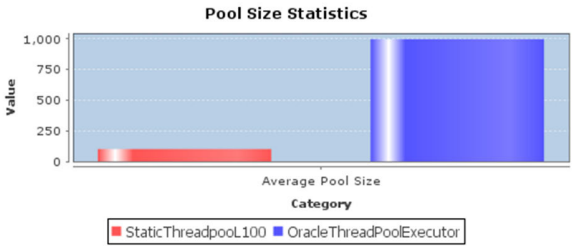



**FIGURE 20.** Comparative analysis of average pool sizes of two different TPSs.

### B. OPERATIONAL FLOW

The working flowchart of PoolRunner is depicted in Figure 21. To test a TPS, the user will first run the server-tier. The server will load the names of available TPS in memory and wait for the client-tier to connect. If a user wants to load a new TPS into the server then he will browse its jar file and load a new TPS in TPS_Service, otherwise he will run the client-tier. Next, the user will connect to the server that will in turn send the names of available TPSs on the server. The User will select some TPS for testing and prepare a test plan in which he will prepare the workload, configure a load generation engine. The user will warm up the TPS on the server side for JIT optimizations and then he will start testing. The load generation engine will send the configured workload with random frequencies. The server-tier will receive requests and submit these requests to the TPS, that will process the requests and handover the responses to the server that will in turn send the responses back to the client-tier. A receiver thread on the client-tier will receive the responses and plot the performance metrics results in a GUI. The user can stop testing any time and he can select any other TPS for either testing it separately or for comparative analysis. In the latter case, the same workload and load generation engine would be utilized automatically, and the user does not need to configure the workload and load generation engine. The client-tier will provide comparative analysis results in the form of graphs and tables.

## V. POOLRUNNER'S VALIDATION

In this section, we will validate the functional and performance accuracy of PoolRunner.

### A. FUNCTIONAL VALIDITY OF POOLRUNNER

To test the functional correctness of PoolRunner, we exercise it with different types of tests. We prove its accuracy by running a series of small to large test pairs to determine how the tool scales. The client and server tiers of PoolRunner run on the localhost. We will first run a series of tests on a simple static TPS and will increase the load in each next test to check the difference. After assessing the static thread pool, we will assess PoolRunner by testing dynamic thread pool. We will also validate the comparative analysis feature of the PoolRunner by comparing both static and dynamic thread pools. Looking at the difference of results in each pair of tests will indicate whether the tool is providing us the accurate results or not. In the server-tier of PoolRunner we have embedded a static TPS (with 10 threads in the pool) and a dynamic TPS (ThreadPoolExecutor [28]). The tests are conducted with constant load distribution to make things simple and to precisely understand the results.

#### 1) FUNCTIONAL TEST#1

The I/O bound requests having service time $\approx$ 100milliseconds are submitted to the server at the rate of 10 requests/second. The load sent to the server is depicted in Figure 22. The test is conducted for 30 seconds.

Since TPS is static, PoolRunner is correctly showing its constant pool size throughout the test run that can be seen in Figure 23.

In every second, every thread picks single request from the queue and run for $\approx$100 milliseconds. Hence, 10 threads produce 10 responses each second as in the Figure 24.

After running the test, the response time statistics are shown in Figure 25, where each request got a proper response from static thread pool. No request must wait in the request queue as request arrival rate is equal to the number of threads in the static TPS. Each request is immediately dequeued from request queue by available thread and completed in time. Hence all percentile response times (PRT) including average response times (ART) are approximately the same. Figure 26 shows a comparison between request rate and throughput. Since ten threads are executing ten tasks every second. The response rate is the same as request arrival rate which we were expecting.

#### 2) FUNCTIONAL TEST#2

In this test, we are increasing the load on static TPS to see the difference in performance. The I/O bound requests having

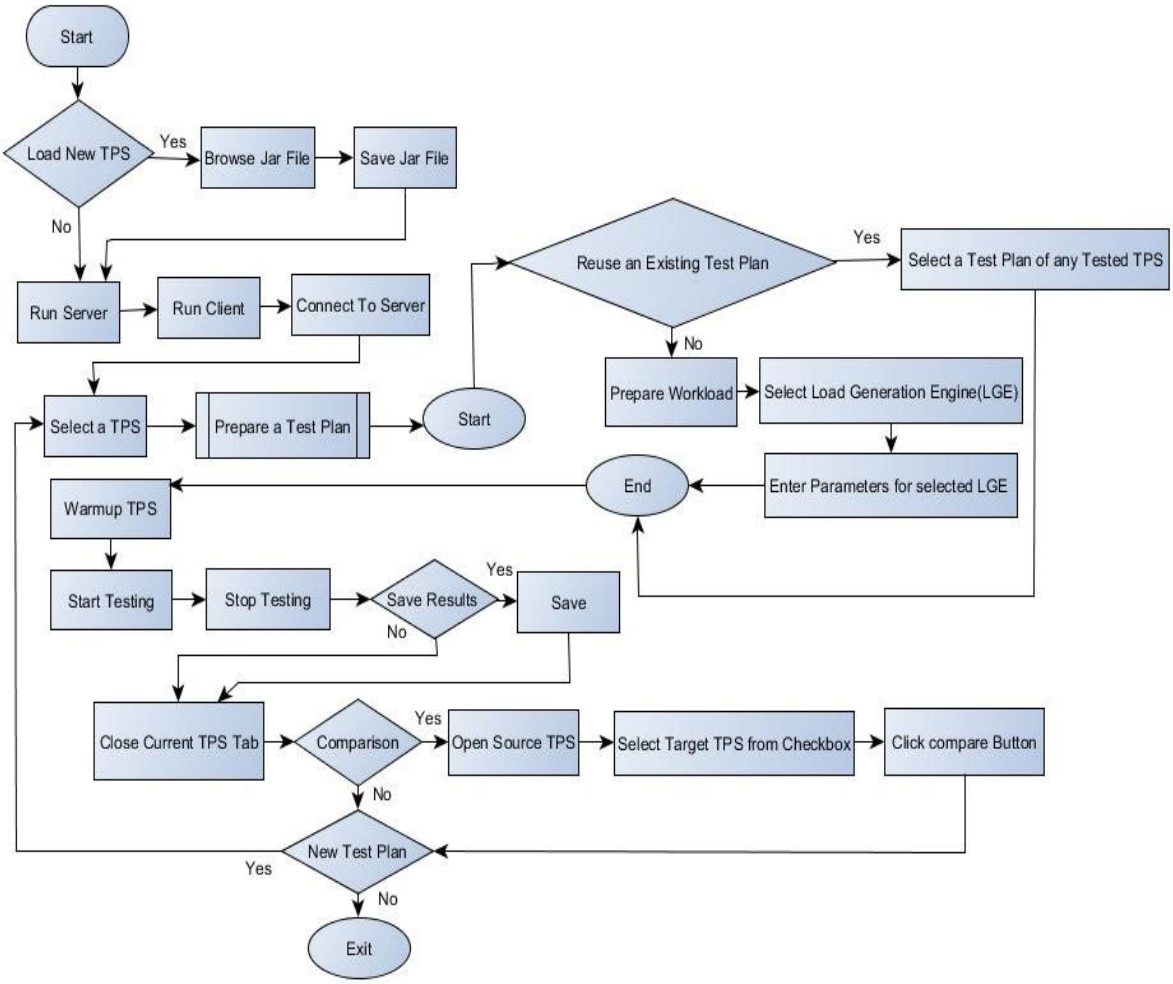



**FIGURE 21.** Operational flowchart of PoolRunner.

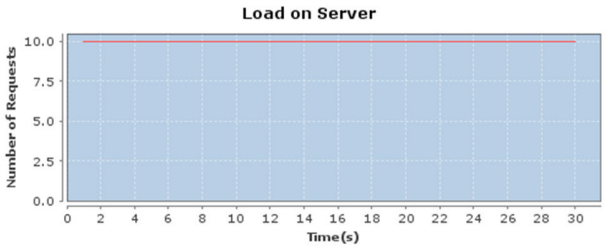



**FIGURE 22.** Load generated by the PoolRunner on TPS.

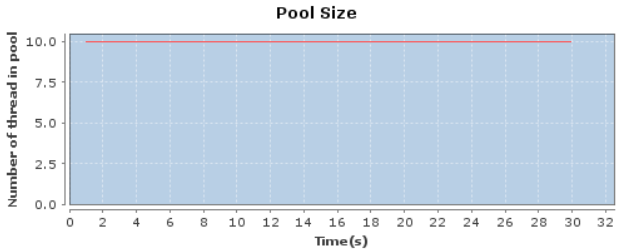



**FIGURE 23.** Quantity of threads inside pool.

service time ≈ 100milliseconds are submitted to the server at the rate of 20 requests/second. The load sent to the server is depicted in Figure 27. The test is conducted for 30 seconds.

In every second, every thread executes 2 requests from the queue as each request run for 100 milliseconds. Hence, 10 threads are producing 20 responses each second as in the Figure 28.

After running the test, the response time statistics in shown in Figure 29, which shows an increase in ART (and percentiles). In every second, 10 requests are immediately picked up by available threads, so their response time will be ≈100ms, while 10 requests must wait for ≈100milliseconds and their response time will be ≈200ms. In the whole run, half of the total requests get immediate response (≈100milliseconds), while the remaining half will have response times of ≈200milliseconds. Hence, the ART is ≈150 (i.e. 146.903). Percentiles data is also accurate because $90^{th}$ and $95^{th}$ percentiles are reaching

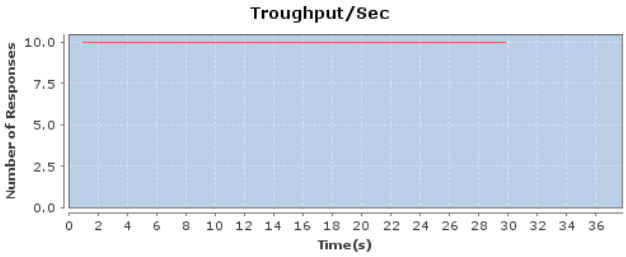



**FIGURE 24.** Responses generated per second by TPS.

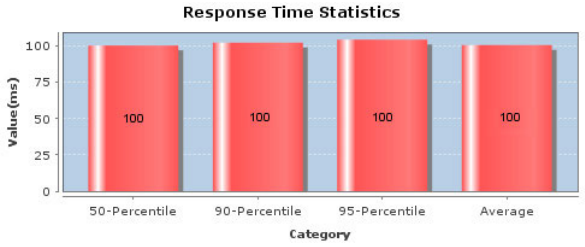



**FIGURE 25.** Response time information.

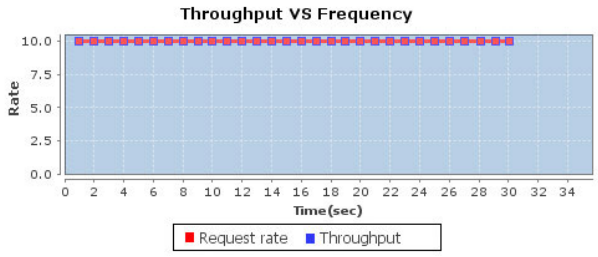



**FIGURE 26.** Comparison of request and response rates.

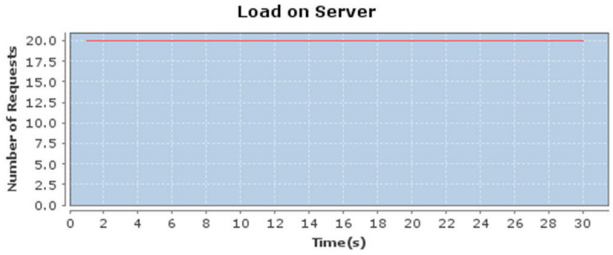



**FIGURE 27.** Request arrival rate on TPS.

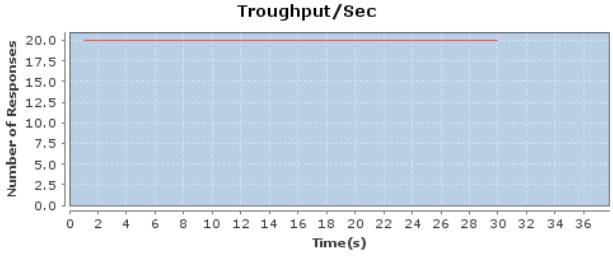



**FIGURE 28.** Responses generated per second by TPS.

200milliseconds, as half of total requests have response time of ≈200milliseconds.




**FIGURE 29.** Response time information.

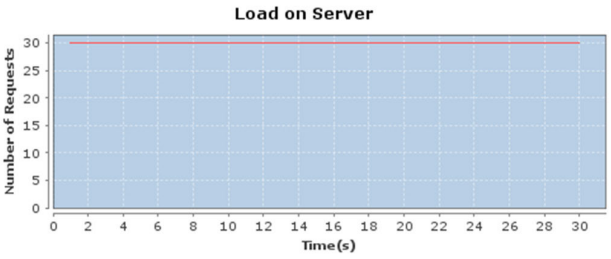



**FIGURE 30.** Request arrival rate on TPS.

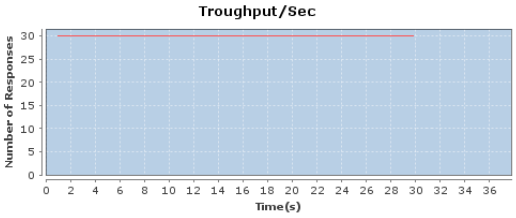



**FIGURE 31.** Responses generated per second by TPS.

### 3) FUNCTIONAL TEST#3

In this test, we are further increasing the load on static TPS, to validate that the PoolRunner is producing accurate outputs.

The I/O bound requests having service time ≈ 100milliseconds are submitted to the server at the rate of 30 requests/second. The load on the server is depicted in Figure 30. The test is conducted for 30 seconds.

In every second, every thread executes 3 requests from the queue because each request run for 100 milliseconds. Hence, 10 threads produce 30 responses each second as in Figure 31.

The response time statistics in Figure 32 show a further increase in the ART (as well as increased percentiles). In every second 10 requests are immediately picked up by available threads and their response time will be ≈100ms, while 10 requests must wait for 100milliseconds and therefore their response time will be ≈200ms and remaining 10 requests wait for ≈200milliseconds and their response time will be ≈300ms. Hence the ART should be around 200ms which can be seen in Figure 32 that it is 195.623. The percentiles data is also accurate because $90^{th}$ and $95^{th}$ percentiles are reaching ≈300milliseconds, as one third of the total requests have response time of ≈300milliseconds.




**FIGURE 32.** Response time information.

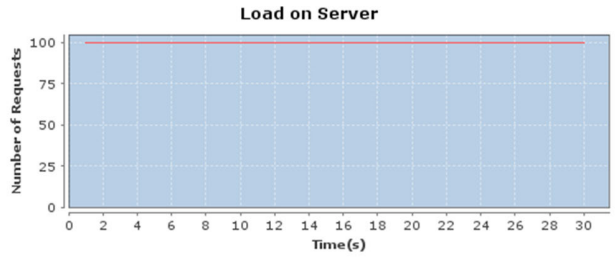



**FIGURE 33.** Request arrival rate on TPS.

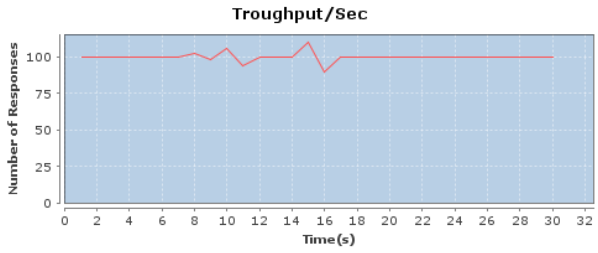



**FIGURE 34.** Responses generated per second by TPS.

#### 4) FUNCTIONAL TEST#4

Finally, we overloaded the static TPS at 100 requests/second and assessed further increase in throughput and response times.

The I/O bound requests having service time ≈ 100milliseconds are submitted to the server at the rate of 100 requests/second. The load sent to the server is depicted in Figure 33. The test is conducted for 30 seconds.

In every second, every thread executes 10 requests from the queue, because each request run for 100milliseconds. Hence, 10 threads produce 100 responses each second as in Figure 34.

The response time statistics in Figure 735 show further increase in ART (as well as increased percentiles). In every second, 10 requests are immediately picked up by available threads and their response time will be around 100ms, while the wait times of remaining requests gradually increases up to 9 second, which can be seen in the figure that ART is 558 while $90^{th}$ and $95^{th}$ percentiles are reaching 1second, as one tenth of total requests have response time of around 1second.

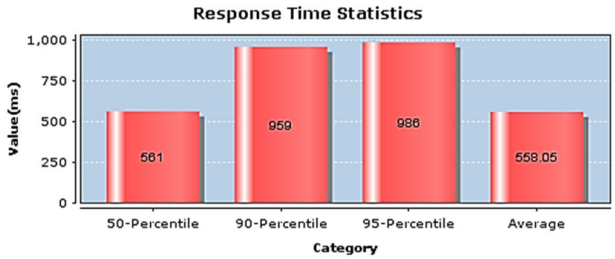



**FIGURE 35.** Response time information.

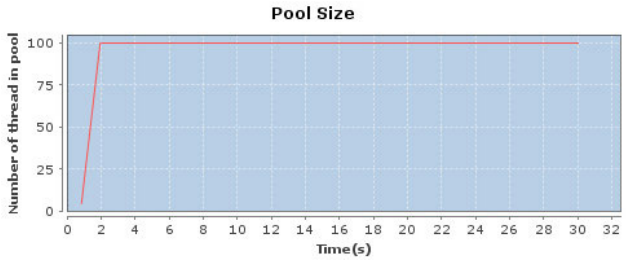



**FIGURE 36.** Quantity of threads inside pool.

#### 5) FUNCTIONAL TEST #5

In this test, we are exercising PoolRunner with a dynamic TPS named ThreadPoolExecutor [28] which is a frequency based TPS that sets its pool size parallel to the request rate. The last test produced 100% throughput with 10 threads, however response times suffered considerably. In this test, we will inspect the response times with the same load as the last test. We will put the load of 100 requests/second on ThreadPoolExecutor, and we expect that the response time will not suffer in this test because ThreadPoolExecutor will create 100 threads for 100 request-rate (for very low I/O intensive load).

The I/O bound requests having service time ≈ 100milliseconds are submitted to the server at the rate of 100 requests/second as shown in the last Figure 33. The test is conducted for 30 seconds.

Figure 36 shows that the pool size increased to 100, because of 100 request rates. ThreadPoolExecutor kept its pool size constant in the whole run as the load probability distribution is constant.

Figure 37 shows improved response times (100 pool size shows ≈100% accuracy in response time). Hence, PoolRunner is giving accurate results.

#### 6) FUNCTIONAL VALIDATION OF COMPARATIVE ANALYSIS GRAPHS

It is also important to validate the comparative analysis graphs generated by PoolRunner. Figure 38 shows the comparison of pool size of Test#4 and Test#5. The graph is showing accurate data of both TPSs at one place, showing pool size of 10 for static TPS, and 100 for ThreadPoolExecutor.

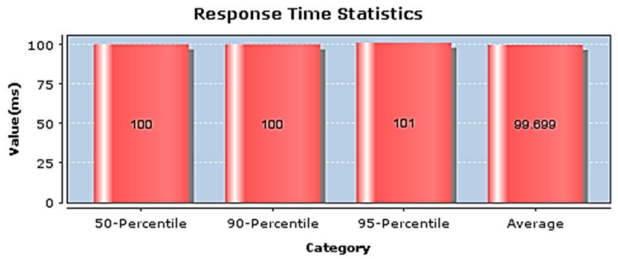



**FIGURE 37.** Response time information.

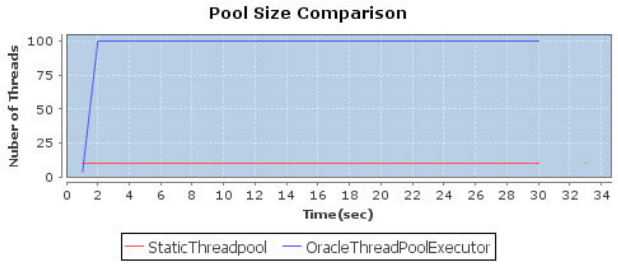



**FIGURE 38.** Pool size comparison between TPSs.

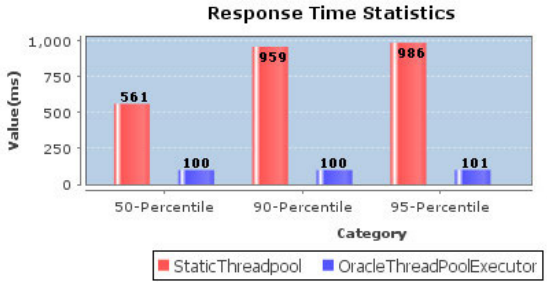



**FIGURE 39.** Response time comparison between TPSs.

Figure 39 shows the comparison of response times between Test#4 and Test#5. This graph also shows accurate data of both TPSs at a single place.

The functional accuracy of PoolRunner has been validated by series of 6 tests and the output graphs are according to the expected output.

### B. STATISTICAL ANALYSIS

This section aims to statistically validate the robustness, accuracy, sensitivity, and scalability of PoolRunner. The validation of 5 functional tests we conducted is performed collectively through statistical analysis. Table 7 is the summary of each test. All tests were conducted on the same workload of ≈100 milliseconds I/O simulated task.

Variance, Confidence interval, and Effect size across all Runs:

Mean (Average) of Average Response Times

Mean=(100+146.903+195.623+558.05+99.6995)/5= 1100.275/5=220.055 ms

$$\text{Variance} = \sum(xi - \bar{x})2/n - 1$$

**TABLE 7.** Summary of functional tests 1 to 5.

| Test | Pool Size | Load | TPS Type | Throughput | ART (ms) | 50th PRT (ms) | 90th PRT (ms) | 95th PRT |
|---|---|---|---|---|---|---|---|---|
| Run 1 | 10 | 10 | Static | 10 | 100 | 100 | 100 | 100 |
| Run 2 | 10 | 20 | Static | 20 | 146.903 | 102 | 196 | 196 |
| Run 3 | 10 | 30 | Static | 30 | 195.623 | 196 | 293 | 294 |
| Run 4 | 10 | 100 | Static | 100 | 558.05 | 561 | 959 | 986 |
| Run 5 | 100 | 100 | Dynamic | 100 | 99.699 | 100 | 100 | 101 |

where:

- Xi is each Average Response Time
- $\bar{x}$ is the mean (220.055 ms)
- n is the number of runs (5)

| Run | xi | $(xi - \bar{x})$ | $(xi - \bar{x})^2$ |
|---|---|---|---|
| 1 | 100 | -120.055 | 14413.22 |
| 2 | 146.903 | -73.152 | 5351.22 |
| 3 | 195.623 | -24.432 | 596.93 |
| 4 | 558.05 | 337.995 | 114259.24 |
| 5 | 99.699 | -120.356 | 14485.51 |

Sum of squares: 14413.22+5351.22+596.93+114259.24 +14485.51=149106.12

Variance= 149106.12/5-1= 37276.53$(ms)^2$

Hence Variance = 37276.53 $(ms)^2$

$$\text{Standard Deviation} = \sqrt{37276.53} \approx 193.08ms$$

Hence Standard Deviation ≈ 193.08 ms

95% Confidence Interval:

Confidence Interval formula (for small samples, n < 30):

$$CI = \bar{x} \pm t_{\alpha/2,n-1} \times \frac{s}{\sqrt{n}}$$

where:

- $\bar{x}$ =220.055
- s=193.08 (standard deviation)
- n=5
- $t_{0.025,4}$ from t-distribution table ≈ 2.776

Margin of Error:

$$ME = 2.776 \times \frac{193.085}{\sqrt{5}} = 2.776 \times 86.32 \approx 239.62$$

Thus, 95% CI:

- Lower bound: 220.055−239.62= −19.565 ms
- Upper bound: 220.055+239.62=459.675

Hence 95% Confidence Interval ≈ [-19.57 ms, 459.68 ms]

Effect Size (Cohen's d between Static and Dynamic)

We'll compare **Run 4 (Static, load 100)** vs **Run 5 (Dynamic, load 100)**:

Formula for Cohen's d:

$$d = \frac{M_1 - M_2}{S_{Pooled}}$$

where

$$S_{Pooled} = \sqrt{\frac{s_1^2 + s_2^2}{2}}$$

- M1 = 558.05 (Static)
- M2 = 99.699M_2 = 99.699M2 = 99.699 (Dynamic)
- Assume that individual variances are 0 (since we only have one value for each).

Thus:

$$d = \frac{558.05 - 99.699}{\sqrt{0}}(undefined)$$

Since no variance inside each group (only 1 sample each), we calculate simplified d:

Simplified d (for single measurements):

$$d = \frac{558.05 - 99.699}{AverageStandardDeviationfromallruns}$$

Use overall s=193.08s = 193.08s=193.08

Thus:

$$d = \frac{458.351}{193.08} \approx 2.37$$

Final Results Summary:

| Metric | Value |
|---|---|
| Mean Average Response Time | 220.055 ms |
| Variance of Average Response Time | 37276.53 (ms)² |
| Metric | Value |
| Standard Deviation | 193.08 ms |
| 95% Confidence Interval | [-19.57 ms, 459.68 ms] |
| Cohen's d (Static vs Dynamic) | ≈ 2.37 (Very Large Effect Size) |

High Variance and Standard Deviation

- Variance = 37,276.53 $ms^2$
- Standard Deviation = 193 ms
- Mean Response Time = ∼220 ms

What It Shows About PoolRunner:

✓ PoolRunner successfully captured large performance fluctuations across different test runs.
✓ These fluctuations are expected when load increases and the static thread pool becomes overloaded.
✓ PoolRunner's ability to track and report this instability accurately reflects its sensitivity and correctness.

Interpretation:

PoolRunner is able to detect and report unstable performance behavior (high variability) when a system is stressed. This validates its reliability for stress testing and bottleneck detection.

Wide Confidence Interval (95% CI: [-19.57 ms, 459.68 ms])

This means the true mean difference in response time (between static and dynamic TPS) could vary in this wide range due to system fluctuations.

What It Shows About PoolRunner:

✓ PoolRunner captures the inherent unpredictability of overloaded systems under different configurations.
✓ Even under such wide behavioral swings, PoolRunner provides statistically sound results using repeated runs.

Interpretation:

PoolRunner is capable of testing both stable and unstable systems. The wide confidence interval reflects PoolRunner's strength in highlighting variability where it naturally exists — a crucial feature for identifying performance risks in TPS configurations.

Very Large Effect Size (Cohen's d ≈ 2.37)

This measures the magnitude of performance improvement from static TPS to dynamic TPS.

What It Shows About PoolRunner:

✓ PoolRunner detected a very large and statistically significant difference between the two configurations.
✓ It shows that the tool is precise and responsive to real performance gains.

Interpretation:

PoolRunner effectively distinguishes major architectural improvements in system design. Its ability to detect strong performance benefits confirms that it is suitable for comparative performance evaluations, especially in real-world dynamic environments.

Conclusion:

All these metrics together confirm that PoolRunner is a robust, scalable, and accurate tool:

✓ It correctly detects response time behavior (mean and percentiles).
✓ It quantifies system instability (variance, CI).
✓ It measures improvement impact (effect size).
✓ It scales with different workloads and thread pool types.

### 1) POOLRUNNER'S PERFORMANCE VALIDATION

In this section, we validate the performance of PoolRunner by examining memory and CPU usage. The PoolRunner's performance is validated in normal and heavy load conditions. The performance of PoolRunner is analyzed by jvisualvm [49], which is a GUI based performance monitoring and troubleshooting tool.

#### *a: PERFORMANCE TEST#1*

In this test we will validate PoolRunner's performance, with a normal load with both I/O bound, and CPU bound workloads.

We performed two sets of tests consecutively, on a static TPS that contains 100 threads in the pool. The workload used in the first test (that is run for 2 minutes) is given in

**TABLE 8.** Workload details.

| Requests | ID | Access frequency | Service Time |
|---|---|---|---|
| 1kb File | 1 | 30% | ≈ 100ms |
| 10kb | 2 | 50% | ≈ 200ms |
| 100kb | 3 | 20% | ≈ 300ms |




**FIGURE 40.** Request arrival pattern on TPS.

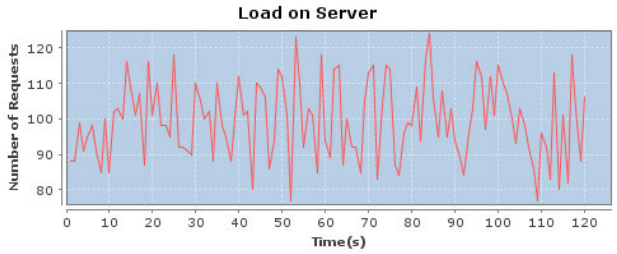



**FIGURE 41.** Request arrival rate pattern on TPS.

Table 8 that is only I/O bound (static requests). The load generated in the first test is depicted in Figure 40, that follows Poisson distribution with $\lambda$ =100. Next, we performed the 2nd test. The workload used in the 2nd test (that is also run for 2minutes) is given in Table 9, that is mixture of CPU (dynamic) and I/O bound(static). The load generated in the 2nd test is depicted in Figure 41, that also follows Poisson distribution with $\lambda$ =100.

Figure 42. shows CPU-Usage in the whole run of two consecutive tests. The CPU-Utilization is low in the first test run, because the workload is I/O bound. It is important to note that the system resources are used effectively by both client and server tiers of PoolRunner as there is no high CPU usage for I/O bound workload. The average CPU usage is 30% in the 2nd test run, as 50% of generated workload is CPU bound. It can be seen in the figure that the CPU utilization is 30% for 2 minutes approximately. What is concluded about CPU-usage is dependent on the type of workload generated by the PoolRunner. In the whole run of 2 tests, which is almost 6 minutes, total garbage collection (GC) activity took only 1.288 seconds that indicates that the code of the PoolRunner is efficient and not generating much garbage that effect CPU-utilization. There are no excessive GC cycles that may interfere with testing and effect response times. There is no code problem in PoolRunner e.g. excessive loops and excessive backend calls.

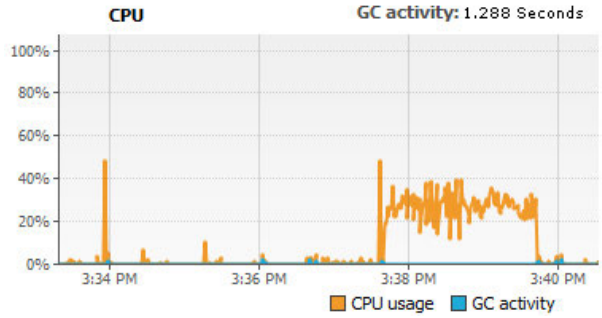



**FIGURE 42.** CPU-usage and garbage collection activity.

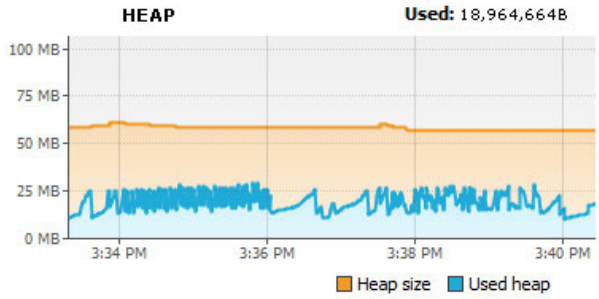



**FIGURE 43.** Steady sawtooth pattern of memory usage.

Figure 43. details the memory utilization during two test runs. The memory usage pattern confirms that there is no memory leakage, as memory is not growing, instead, it is a flat baseline trend in the form of double-sawtooth pattern, where memory is steadily increasing and decreasing due to the small GC pauses which are necessary for reclaiming memory by GC. The small sawtooth pattern is due to the memory allocation and deallocation, where memory is being allocated to objects and garbage collected when it is not being referenced. The steady sawtooth pattern is an indication of healthy memory utilization that is not under stress [50].

### *b: PERFORMANCE TEST#2*

In this test, we validate PoolRunner's performance in a heavy load condition, by undertaking dynamic thread pool i.e. ThreadPoolExecutor. We performed two tests consecutively on ThreadPoolExecutor. We run each test for approximately 5 minutes, and the whole test run for PoolRunner assessment is approximately 10 minutes. The workload used in the first test is given in Table 9 that is only I/O bound (static requests). The load generated in the first test is depicted in Figure 44, that follows Poisson distribution with $\lambda$ =1000. Next, we performed the 2nd test. The workload used in the 2nd test is given in Table 10, that is mixture of CPU (dynamic) and I/O bound(static). The load generated in the 2nd test is depicted in Figure 45, that also follows Poisson distribution with $\lambda$ =1000. We kept a low percentage of CPU intensive workload because this is a heavy load test on a local host.

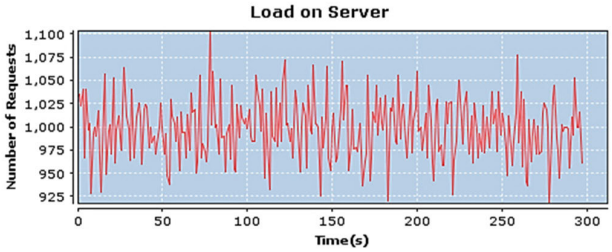



**FIGURE 44.** Request rate arrival pattern.

**TABLE 9.** Workload detail.

| Requests | ID | Access frequency | Service Time |
|---|---|---|---|
| 1kb File | 1 | 50% | ≈ 100ms |
| low CPU-intensive task | 2 | 50% | ≈ 40ms |




**FIGURE 45.** Request rate arrival pattern on the server.

Figure 46. shows CPU-Usage in the whole run of 2 consecutive tests. CPU-Utilization is low in the first test run (for 5 minutes) because the workload is I/O bound. It is important to notice that system resources are used effectively by both client and server tiers of PoolRunner as there is no high CPU usage for I/O bound workload. CPU usage is approximately 50% in the 2nd test run, as there are 10% CPU intensive requests sent every second. Since the load is generated with the rate of 1000 requests/second, there are around 100 CPU-intensive requests sent every second. The result of high CPU workload kept CPU usage high as compared to previous tests. In the whole run of 2 tests which is almost 10 minutes(approximately), total garbage collection (GC) activity took only 4.761 seconds that specifies that code of PoolRunner is not generating much garbage. There are no excessive GC cycles that may interfere with testing and effect response times.

Figure 47. details memory utilization during two test runs of 10 minutes (approximately). The memory usage pattern in the figure confirms that there is no memory leakage as memory is not growing all the time, instead memory is steadily increasing and decreasing due to small GC pauses which are necessary for reclaiming memory by GC. Unlike the previous test, the sawtooth spikes are high due to heavy workload on TPS. Like previous performance tests, the sawtooth pattern is due to reclaiming memory by GC.

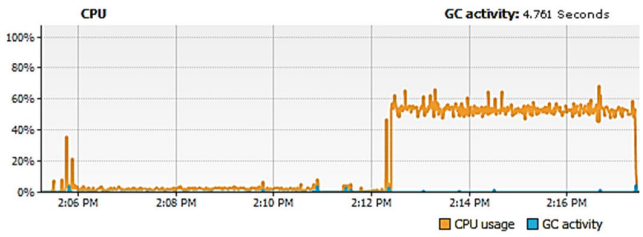



**FIGURE 46.** CPU usage and GC activity in whole test run.

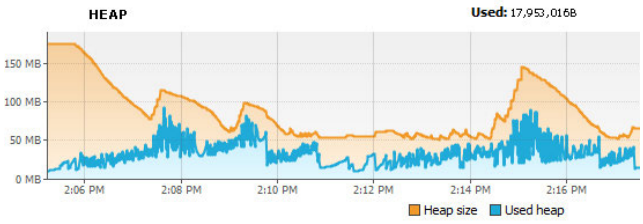



**FIGURE 47.** Steady sawtooth pattern of memory usage.

In this heavy load test, there is not out of memory pattern in the figure, nor excessive GC activity, which shows that PoolRunner is performance efficient in terms of resource utilization in high load situations.

## VI. CONCLUSION AND FUTURE WORK

This paper presented a lightweight first client-server based extensible performance testing simulation tool for Thread Pool Systems (TPS) named PoolRunner, developed in JAVA. Unlike existing benchmarking and load testing tools, PoolRunner allows researchers and practitioners to simulate open-system workloads, measure internal TPS metrics in real time, and compare multiple TPS variants within a single framework along with extensibility.

Through functional validation, PoolRunner was tested under multiple workload intensities using both static and dynamic TPS configurations. The results confirmed the correctness of its internal metrics collection (e.g., response times, wait latencies, pool sizes), its support for comparative performance analysis, and its accuracy in producing expected behavioral outcomes under stress. Statistical analysis across test runs further validated PoolRunner's robustness and sensitivity. The tool captured performance fluctuations with high variance and wide confidence intervals, while also detecting significant effect size improvements when dynamic TPS was used. This demonstrates its suitability not only for functional testing, but also for rigorous comparative performance evaluations. Additionally, PoolRunner proved efficient in terms of system resource usage. Performance profiling with jvisualvm showed minimal garbage collection activity, stable memory patterns, and predictable CPU utilization—even under high-load, mixed workloads. This validates PoolRunner's ability to operate efficiently in long simulation runs without introducing overhead or instability.

Overall, PoolRunner fills a notable gap in TPS performance research by offering a reusable, extensible, and metrics-aware platform for testing thread pool systems in both academic and industrial settings.

In the future, we will present the distributed form of PoolRunner that may be operated from multiple nodes (with a single dashboard) to generate a more powerful load to overload the server. The next generation of this tool will leverage AI-based technologies to enhance its functionality including AI-based load generation to overload the server etc. Moreover, we will address PoolRunner's interoperability issues to enable seamless integration and testing of thread pools developed in other languages including Python, C++, and Scala. Furthermore, we will develop mechanisms in the tool that can integrate publicly available workload data sets to simulate real-world traffic and overload a TPS.

## VII. LIMITATIONS OF THE PROPOSED STUDY

While statistical analysis (e.g., variance, confidence intervals, effect sizes) has been conducted, it is currently performed externally. The tool does not yet automate or embed real-time statistical summary generation in the dashboard or output files. The current SPI interface is Java-based, which means TPSs written in other programming languages (e.g., Python, C++, Scala) cannot yet be directly embedded into the framework. It does not yet support adaptive workload shaping, user-defined load patterns, or trace-based replay during runtime.

## DATA AND CODE AVAILABILITY STATEMENT

The code related to the simulation tool presented in the proposed study is available at the following link:**https://github.com/faisalbahadurhu/jpoolrunner**

## CONFLICTS OF INTEREST

The authors declare that they have no conflicts of interest.

## REFERENCES

[1] M. E. Haque, M. Asikuzzaman, I. U. Khan, I.-H. Ra, M. S. Hossain, and S. B. H. Shah, "Comparative study of IoT-based topology maintenance protocol in a wireless sensor network for structural health monitoring," *Remote Sens.*, vol. 12, no. 15, p. 2358, Jul. 2020, doi: 10.3390/rs12152358.

[2] V. V. Kumar, U. Padmavathi, C. P. Ranjith, J. Balaji, and C. N. S. V. Kumar, "An elixir for blockchain scalability with channel based clustered sharding," *Scalable Comput., Pract. Exp.*, vol. 25, no. 2, pp. 997–1004, Feb. 2024, doi: 10.12694/scpe.v25i2.2441.

[3] T. Maqsood, O. Khalid, R. Irfan, S. A. Madani, and S. U. Khan, "Scalability issues in online social networks," *ACM Comput. Surveys*, vol. 49, no. 2, pp. 1–42, Jun. 2017, doi: 10.1145/2968216.

[4] G. James, E. Anietie, E. Abraham, E. Oduobuk, and P. Okafor, "Analysis of support vector machine and random forest models for predicting the scalability of a broadband network," *J. Nigerian Soc. Phys. Sci.*, vol. 2024, p. 2093, Jul. 2024, doi: 10.46481/jnsps.2024.2093.

[5] O. Mjihil, D. Huang, and A. Haqiq, "Improving attack graph scalability for the cloud through SDN-based decomposition and parallel processing," in *Proc. 3rd Int. Symp. Ubiquitous Netw.*, 2017, pp. 193–205, doi: 10.1007/978-3-319-68179-5_17.

[6] M. Ali, A. I. Jehangiri, O. I. Alramli, Z. Ahmad, R. M. Ghoniem, M. A. Ala'anzy, and R. Saleem, "Performance and scalability analysis of SDN-based large-scale Wi-Fi networks," *Appl. Sci.*, vol. 13, no. 7, p. 4170, Mar. 2023, doi: 10.3390/app13074170.

[7] Y. Zhang, X. Wang, Y. Gang, J. Wang, S. Wu, P. Zhang, and Y. Shi, "6G SAGIN information transmission model," *IEEE Commun. Mag.*, early access, Jan. 6, 2025, doi: 10.1109/MCOM.001.2400351.

[8] G. Denaro, A. Polini, and W. Emmerich, "Early performance testing of distributed software applications," in *Proc. 4th Int. Workshop Softw. Perform.*, Jan. 2004, pp. 94–103, doi: 10.1145/974044.974059.

[9] G. Cheng, J. Xia, L. Luo, H. Mi, D. Guo, and R. T. B. Ma, "HyperPart: A hypergraph-based abstraction for deduplicated storage systems," *IEEE Trans. Cloud Comput.*, vol. 13, no. 1, pp. 46–60, Jan. 2025, doi: 10.1109/TCC.2024.3502464.

[10] J. Zhang, M. Ma, P. Wang, and X.-D. Sun, "Middleware for the Internet of Things: A survey on requirements, enabling technologies, and solutions," *J. Syst. Archit.*, vol. 117, Aug. 2021, Art. no. 102098, doi: 10.1016/j.sysarc.2021.102098.

[11] M. Gillani, H. A. Niaz, and A. Ullah, "Integration of software architecture in requirements elicitation for rapid software development," *IEEE Access*, vol. 10, pp. 56158–56178, 2022, doi: 10.1109/ACCESS.2022.3177659.

[12] J. Hu, H. Jiang, Z. Xiao, S. Chen, S. Dustdar, and J. Liu, "HeadTrack: Real-time human–computer interaction via wireless earphones," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 4, pp. 990–1002, Apr. 2024, doi: 10.1109/JSAC.2023.3345381.

[13] Y. Xu, L. Ding, P. He, Z. Lu, and J. Zhang, "Meta: A memory-efficient tri-stage polynomial multiplication accelerator using 2D coupled-BFUs," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 2, pp. 647–660, Feb. 2025, doi: 10.1109/TCSI.2024.3461736.

[14] M. García-Valls, "A proposal for cost-effective server usage in CPS in the presence of dynamic client requests," in *Proc. IEEE 19th Int. Symp. Real-Time Distrib. Comput. (ISORC)*, May 2016, pp. 19–26, doi: 10.1109/ISORC.2016.13.

[15] M. García-Valls and C. Calva-Urrego, "Improving service time with a multicore aware middleware," in *Proc. Symp. Appl. Comput.*, Apr. 2017, pp. 1548–1553, doi: 10.1145/3019612.3019741.

[16] F. Azzedin and K. Al-Issa, "Handling input/output operations in Web servers: Issues and countermeasures," *Concurrency Comput., Pract. Exp.*, vol. 24, no. 7, pp. 699–710, May 2012, doi: 10.1002/cpe.1912.

[17] K. Abirami and N. Harini, "A hybrid approach to dynamic thread allocation driven by workload and task prioritization," *Social Netw. Comput. Sci.*, vol. 6, no. 3, p. 212, Feb. 2025, doi: 10.1007/s42979-025-03704-z.

[18] W. Ye, Y. Tong, and C. Cao, "A Kalman filter based hill-climbing strategy for application server configuration," in *Proc. IEEE SmartWorld, Ubiquitous Intell. Comput., Adv. Trusted Comput., Scalable Comput. Commun., Cloud Big Data Comput., Internet People Smart City Innov. (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI)*, Oct. 2018, pp. 1524–1531, doi: 10.1109/SMARTWORLD.2018.00263.

[19] Z. Wang, P. Subedi, M. Dorier, P. E. Davis, and M. Parashar, "Staging based task execution for data-driven, in-situ scientific workflows," in *Proc. IEEE Int. Conf. Cluster Comput. (CLUSTER)*, Sep. 2020, pp. 209–220, doi: 10.1109/CLUSTER49012.2020.00031.

[20] B. Shoshany, "A C++17 thread pool for high-performance scientific computing," *SoftwareX*, vol. 26, May 2024, Art. no. 101687, doi: 10.1016/j.softx.2024.101687.

[21] W. Yang, M. Ao, J. Sun, G. Wang, Y. Li, C. Li, and Z. Shao, "Adaptive parallel scheduling scheme for smart contract," *Mathematics*, vol. 12, no. 9, p. 1347, Apr. 2024, doi: 10.3390/math12091347.

[22] J. Han, G. Zhu, E. Lee, and Y. Son, "Design and implementation of an efficient and parallel bad block checker for parallelism of storage devices," *Cluster Comput.*, vol. 26, no. 5, pp. 2615–2627, Oct. 2023, doi: 10.1007/s10586-021-03353-w.

[23] F. Bahadur, A. Iqbal Umar, I. Ullah, F. Algarni, M. Asghar Khan, and S. M. Mostafa, "A perfect knob to scale thread pool on runtime," *Comput., Mater. Continua*, vol. 72, no. 1, pp. 1483–1493, 2022, doi: 10.32604/cmc.2022.024936.

[24] F. Bahadur, A. Iqbal Umar, I. Ullah, F. Algarni, and M. Asghar Khan, "The double edge sword based distributed executor service," *Comput. Syst. Sci. Eng.*, vol. 42, no. 2, pp. 589–604, 2022, doi: 10.32604/csse.2022.022319.

[25] D. Casini, "A theoretical approach to determine the optimal size of a thread pool for real-time systems," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2022, pp. 66–78, doi: 10.1109/RTSS55097.2022.00016.

[26] I. V. Stetsenko and O. Dyfuchyna, "Thread pool parameters tuning using simulation," in *Proc. Adv. Comput. Sci. Eng. Educ.*, 2020, pp. 78–89, doi: 10.1007/978-3-030-16621-2_8.

[27] G. Somashekar and A. Gandhi, "Towards optimal configuration of microservices," in *Proc. 1st Workshop Mach. Learn. Syst.*, Apr. 2021, pp. 7–14, doi: 10.1145/3437984.3458828.
[28] D. L. Freire, R. Z. Frantz, and F. R. Frantz, "Towards optimal thread pool configuration for run-time systems of integration platforms," *Int. J. Comput. Appl. Technol.*, vol. 62, no. 2, p. 129, 2020, doi: 10.1504/ijcat.2020.104692.
[29] N. Costa, M. Jayasinghe, A. Atukorale, S. Abeysinghe, S. Perera, and I. Perera, "ADAPT-T: An adaptive algorithm for auto-tuning worker thread pool size in application servers," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jun. 2019, pp. 1–6, doi: 10.1109/ISCC47284.2019.8969737.
[30] C. Chen, J. Cui, G. Qu, and J. Zhang, "Write+sync: Software cache write covert channels exploiting memory-disk synchronization," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8066–8078, 2024, doi: 10.1109/TIFS.2024.3414255.
[31] G. Sun, G. Zhu, D. Liao, H. Yu, X. Du, and M. Guizani, "Cost-efficient service function chain orchestration for low-latency applications in NFV networks," *IEEE Syst. J.*, vol. 13, no. 4, pp. 3877–3888, Dec. 2019, doi: 10.1109/JSYST.2018.2879883.
[32] G. Sun, Y. Li, D. Liao, and V. Chang, "Service function chain orchestration across multiple domains: A full mesh aggregation approach," *IEEE Trans. Netw. Service Manage.*, vol. 15, no. 3, pp. 1175–1191, Sep. 2018, doi: 10.1109/TNSM.2018.2861717.
[33] P. Wang, W. Song, H. Qi, C. Zhou, F. Li, Y. Wang, P. Sun, and Q. Zhang, "Server-initiated federated unlearning to eliminate impacts of low-quality data," *IEEE Trans. Services Comput.*, vol. 17, no. 3, pp. 1196–1211, May 2024, doi: 10.1109/TSC.2024.3355188.
[34] T. Li, Y. Li, M. Zhang, S. Tarkoma, and P. Hui, "You are how you use apps: User profiling based on spatiotemporal app usage behavior," *ACM Trans. Intell. Syst. Technol.*, vol. 14, no. 4, pp. 1–21, Aug. 2023, doi: 10.1145/3597212.
[35] Oracle. *Class Thread Pool Executor*. Accessed: Feb. 10, 2025. [Online]. Available: https://docs.oracle.com/javase/7/docs/api/java/util/concurrent/ThreadPoolExecutor.html
[36] H. Samoaa and P. Leitner, "An exploratory study of the impact of parameterization on JMH measurement results in open-source projects," in *Proc. ACM/SPEC Int. Conf. Perform. Eng.*, Apr. 2021, pp. 213–224, doi: 10.1145/3427921.3450243.
[37] Openjdk. *Code Tools: JCStress*. Accessed: Feb. 10, 2025. [Online]. Available: https://openjdk.java.net/projects/code-tools/jcstress
[38] R. Han, L. K. John, and J. Zhan, "Benchmarking big data systems: A review," *IEEE Trans. Services Comput.*, vol. 11, no. 3, pp. 580–597, May 2018, doi: 10.1109/TSC.2017.2730882.
[39] A. Kusum, I. Neamtiu, and R. Gupta, "Safe and flexible adaptation via alternate data structure representations," in *Proc. 25th Int. Conf. Compiler Construct.*, Mar. 2016, pp. 34–44, doi: 10.1145/2892208.2892220.
[40] R. Abbas, Z. Sultan, and S. N. Bhatti, "Comparative analysis of automated load testing tools: Apache JMeter, Microsoft visual studio (TFS), LoadRunner, siege," in *Proc. Int. Conf. Commun. Technol. (ComTech)*, Apr. 2017, pp. 39–44, doi: 10.1109/COMTECH.2017.8065747.
[41] A. Kołtun and B. Pańczyk, "Comparative analysis of Web application performance testing tools," *J. Comput. Sci. Inst.*, vol. 17, pp. 351–357, Dec. 2020, doi: 10.35784/jcsi.2209.
[42] D. Rahmel, "Testing a site with ApacheBench, JMeter, and selenium," in *Advanced Joomla!*. Berkeley, CA, USA: Apress, 2013, pp. 211–247, doi: 10.1007/978-1-4302-1629-2_9.
[43] L. Mamushiane, A. A. Lysko, and S. Dlamini, "A comparative evaluation of the performance of popular SDN controllers," *IFIP Wireless Days*, vol. 2018, pp. 54–59, Apr. 2018, doi: 10.1109/wd.2018.8361694.
[44] Z. Ahmad, A. I. Jehangiri, M. A. Ala'anzy, M. Othman, R. Latip, S. K. U. Zaman, and A. I. Umar, "Scientific workflows management and scheduling in cloud computing: Taxonomy, prospects, and challenges," *IEEE Access*, vol. 9, pp. 53491–53508, 2021, doi: 10.1109/ACCESS.2021.3070785.
[45] X. Zou, J. Yuan, P. Shilane, W. Xia, H. Zhang, and X. Wang, "From hyper-dimensional structures to linear structures: Maintaining deduplicated data's locality," *ACM Trans. Storage*, vol. 18, no. 3, pp. 1–28, Aug. 2022, doi: 10.1145/3507921.
[46] Oracle. *The JavaTM Tutorials*. Accessed: Feb. 10, 2025. [Online]. Available: https://docs.oracle.com/javase/tutorial/ext/basics/spi.html
[47] M. Andreolini, V. Cardellini, and M. Colajanni, "Benchmarking models and tools for distributed Web-server systems," in *Proc. IFIP Int. Symp. Comput. Perform. Modeling, Meas. Eval.*, 2002, pp. 208–235, doi: 10.1007/3-540-45798-4_10.
[48] A. Godbole, S.-Y. Kim, R. Guzman, and W. W. Smari, "Performance evaluation studies of client-server models using SPEC Web99 benchmarks," in *Proc. 5th IEEE Workshop Mobile Comput. Syst. Appl.*, Jul. 2003, pp. 406–414, doi: 10.1109/IRI.2003.1251444.
[49] S. O. Fageeri, R. Ahmad, and H. Alhussian, "A performance analysis of association rule mining algorithms," in *Proc. 3rd Int. Conf. Comput. Inf. Sci. (ICCOINS)*, Aug. 2016, pp. 328–333, doi: 10.1109/ICCOINS.2016.7783236.
[50] Z. Budimlić and K. Kennedy, "Optimizing Java: Theory and practice," *Concurr. Pract. Exp.*, vol. 9, no. 6, pp. 445–463, Jun. 1997, doi: 10.1002/(sici)1096-9128(199706)9:6<445::aid-cpe301>3.0.co;2-l.



**FAISAL BAHADUR** received the Ph.D. degree in computer science from Hazara University, Mansehra, Pakistan. He has an extensive experience of research and teaching with the Computer Science and Information Technology Department. Currently, he is an Assistant Professor with the CS & IT Department, Hazara University. His research interests include performance testing, simulation and modeling, mobile application optimization, concurrency and parallelism, cloud and edge computing, smart cities infrastructures, and the Internet of Things (IoT).



**ZULFIQAR AHMAD** received the M.Sc. degree in computer science (CS) from Hazara University, Mansehra, Pakistan, in 2012, the M.S. degree in CS from COMSATS University, Abbottabad, Pakistan, in 2016, and the Ph.D. degree in CS from the Department of Computer Science and Information Technology, Hazara University, in 2022. He is the author of several publications in the field of fog computing, cloud computing, high performance computing, and scientific workflows execution and management. His research interests include scientific workflow management in cloud computing, the Internet of Things, fog computing, edge computing, cybersecurity, and wireless sensor networks (WSNs).



**ABDULMOHSEN ALGARNI** received the Ph.D. degree from Queensland University of Technology, Australia, in 2012. He was a Research Associate with the School of Electrical Engineering and Computer Science, Queensland University of Technology, in 2012. He is currently a Professor with the College of Computer Science, King Khalid University. His research interests include artificial intelligence, data mining, text mining, machine learning, information retrieval, and information filtering.